

Identidades en Fabric: PKI, Certificados, CAs y MSP

De la criptografía asimetrica al control de acceso en consorcios

Lo que vas a aprender

1. ¿Qué es un certificado y para qué sirve?
2. ¿Quién es la autoridad certificadora (CA) y por qué confiamos en ella?
3. ¿Qué es un certificado raíz y qué significa la cadena de confianza?
4. ¿Por qué Fabric usa DOS CAs distintas (identidad y TLS)?
5. ¿Qué es el MSP y cómo se materializa físicamente?
6. ¿Cómo se valida una transaccion paso a paso?

1. ¿Qué es un certificado?

Criptografía asimetrica + identidad firmada

Repaso: criptografía asimétrica

Cada usuario tiene un par de claves criptográficamente ligadas:

- Clave privada — secreta, nunca se comparte

- Clave pública — derivada de la privada, se comparte libremente

Propiedades:

- Lo cifrado con la pública solo se descifra con la privada

- Lo firmado con la privada se verifica con la pública

- De la pública NO se puede deducir la privada (en tiempo razonable)

Ejemplo cotidiano:

- DNI electrónico, certificado de firma de la FNMT

- TLS de las webs (<https://>) usa exactamente este mismo principio

Problema: ¿cómo sabemos a quién pertenece una clave pública?

Cualquiera puede generar un par de claves y reclamar ser quien quiera

Si Alice genera claves y dice 'soy el Banco Santander', ¿quién la cree?

Necesitamos un mecanismo para asociar una clave pública con una identidad real

La solución: una entidad de confianza firma esa asociación

Esa firma se materializa en un documento llamado 'certificado'

El certificado X.509: definicion

Un certificado es un documento digital firmado que ASOCIA:

- Una identidad (nombre, email, organizacion, atributos)

- Una clave pública (la del titular)

- Un periodo de validez (fecha de inicio y caducidad)

- Un identificador unico (serial number)

Y todo eso firmado por una autoridad de confianza (la CA)

X.509 es el estandar internacional (RFC 5280) que define el formato

Es lo que usa toda Internet: TLS/HTTPS, DNI electronico, Fabric, Bitcoin (parcialmente)...

Estructura de un certificado X.509

Subject: a quien identifica (CN=peer0.org1, O=Org1)

Issuer: quién lo emite (la CA)

Validity: desde cuando hasta cuando es valido

Subject Public Key: la clave pública del titular

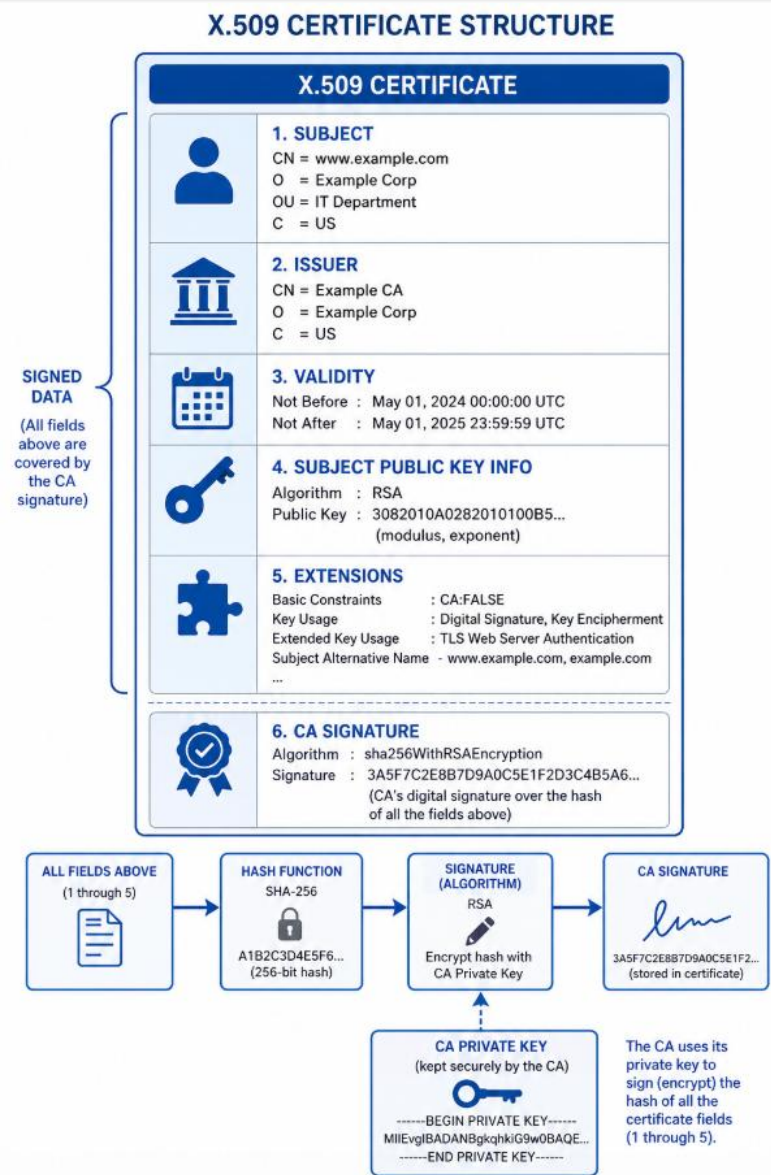
Extensions: atributos custom, SANs (alt names), Key Usage...

Signature: firma de la CA sobre todo lo anterior

La firma de la CA es lo que da validez al certificado.

Sin firma valida, el certificado no vale nada.

Anatomía de un certificado X.509



2. La Autoridad Certificadora (CA)

Quién emite los certificados y por qué confiamos en ella

¿Qué es una CA?

Certificate Authority — autoridad que emite certificados

Es la entidad que firma los certificados de los usuarios finales

Su firma es lo que convierte un par de claves en una identidad reconocida

Analogía: el registro civil emite los DNIs. Su sello hace que un DNI sea válido.

Sin esa autoridad, cualquiera podría fabricar su propio 'DNI' y nadie lo creería.

Tipos de CA según ámbito

CAs publicas (mundo abierto):

Let's Encrypt, DigiCert, GlobalSign, Sectigo...

Su raíz esta preinstalada en navegadores y sistemas operativos

Cualquiera puede pedir un cert (con verificación)

Caso: HTTPS de tu web

CAs privadas (organizaciones internas):

Una empresa monta su propia CA para certificados internos

VPN, intranet, sistemas internos

Solo sus empleados confian en esa CA

CAs de consorcio (entre orgs que se conocen):

Cada miembro del consorcio opera SU propia CA

Las orgs intercambian sus certs raíz publicos para confiar mutuamente

Caso: Hyperledger Fabric (Fabric CA)

¿Por qué confiamos en una CA?

Por una de tres razones:

Tecnológicamente: la firma criptografica es matematicamente verificable

Pero la decision de confiar en CA es una decision HUMANA, no técnica

Problema clave: si la CA es comprometida...

El atacante puede emitir certificados falsos firmados por la CA

Esos certificados son técnicamente validos — pasan la verificación

Pueden suplantar identidades, firmar transacciones fraudulentas

Caso real: en 2011 hackearon DigiNotar (una CA de Holanda)

- emitieron certs falsos de google.com
- se usaron para espiar a usuarios iranies
- DigiNotar quebro y fue eliminada de los navegadores

Por eso la clave privada de la CA es el activo MAS critico.

En produccion seria, suele estar offline (HSM, bovedas) y se usa lo minimo posible.

3. Certificado raíz y cadena de confianza

El árbol de la confianza

¿Qué es un certificado raíz?

Es el certificado original de la CA — la 'raíz' del árbol de confianza

Característica única: es AUTO-FIRMADO

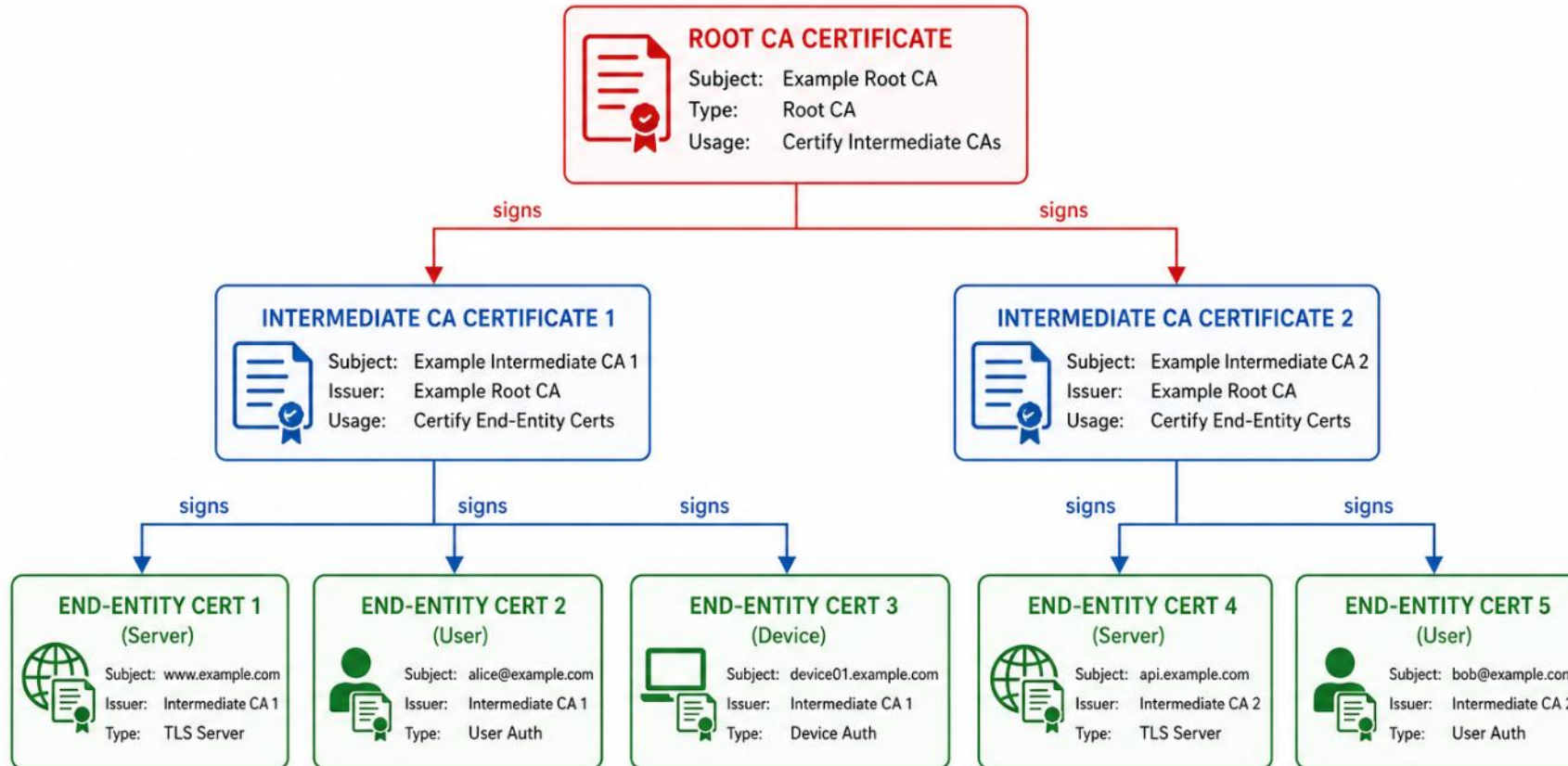
- el Subject y el Issuer son la misma entidad (la CA)
- es el unico cert que no necesita otro cert por encima

Otros nombres: root cert, trust anchor (ancla de confianza), CA cert

Es el punto de partida de toda la cadena. Si confias en el raíz, puedes confiar en todo lo que de el deriva.

El árbol de certificados

CERTIFICATE TRUST CHAIN



- Root CA Certificate (self-signed) – Trust anchor
- Intermediate CA Certificates – Signed by parent (Root CA)
- End-Entity Certificates – Signed by issuing Intermediate CA



Trust flows from top to bottom.
Each certificate is signed by its parent.
End-entity certificates are trusted because the chain links up to the trusted Root CA.

Cadena de confianza (chain of trust)

Es la secuencia de certificados que conecta a un titular con la raíz

Cada cert es firmado por el cert que esta encima

Para validar, recorres la cadena hasta llegar al raíz

Ejemplo de validación de un cert de Alice:

Validación paso a paso

Tienes el cert de Alice y quieres validarlo:

1. ¿La firma del cert de Alice es valida usando la clave pública del Intermedio 1?
2. ¿La firma del cert del Intermedio 1 es valida usando la clave pública del Raíz?
3. ¿Conoces el cert Raíz? ¿Esta en tu lista de certs de confianza?

Si todas las respuestas son SI -> el cert de Alice es valido

Si alguna falla -> el cert se rechaza

Importante: el verificador SOLO necesita tener el cert RAIZ

— no necesita tener todos los certs de Alice, Bob, Carlos, etc.

¿Por qué hay certs intermedios?

Para PROTEGER el cert raíz

El raíz es el activo mas crítico — comprometerlo seria catastrofico

Solucion: el raíz emite un cert intermedio y se guarda offline

El intermedio es el que esta online y emite los certs de usuarios

Si comprometen el intermedio:

- Se revoca y se emite uno nuevo desde el raíz
- Solo afecta a los certs emitidos por ese intermedio

Si comprometieran el raíz:

- Hay que regenerar TODA la cadena, todos los miembros vuelven a empezar

Por eso en producción seria es habitual ver cadenas de 2-3 niveles.

En Fabric con cryptogen vs producción

Cryptogen (desarrollo y aula):

- Una sola CA raíz por org

- Esa raíz firma directamente todos los certs (peers, admins, users)

- Sin intermedios — cadena de 1 nivel

- Simple pero menos seguro

Producción seria:

- CA raíz offline en HSM o bóveda

- CA intermedia online, firma certs de usuarios

- Cadena típica de 2-3 niveles

- Si comprometen el intermedio, se revoca sin afectar a la raíz

En 2011, una CA holandesa fue hackeada y emitio cientos de certificados falsos.
Entre ellos un cert de google.com que se uso para espiar a 300.000 usuarios iranies.
El daño fue tal que la CA quebro y fue eliminada de la lista de CAs de confianza.

Misión: investigar y responder:

1. ¿Como se llamaba esa CA holandesa?
2. ¿Cuantos certificados falsos emitieron exactamente?
3. ¿Que medidas tomaron Mozilla, Microsoft y Google despues del hack?
4. ¿Que es 'Certificate Transparency' y como surgio como respuesta a este caso?

Tiempo: 15 minutos. Es un caso fascinante de seguridad informatica.

Treasure Hunt: respuestas

1. La CA se llamaba DigiNotar — una empresa holandesa que tambien emitia certs para el gobierno de Paises Bajos (PKloverheid).
2. Emitieron al menos 531 certificados fraudulentos. Entre ellos:
*.google.com, *.facebook.com, *.skype.com, *.cia.gov, *.mozilla.org
3. Mozilla, Microsoft y Google retiraron a DigiNotar de sus stores de confianza.
El gobierno holandés intervino la empresa, que quebro en septiembre 2011.
4. Certificate Transparency (CT) es un sistema publico donde TODAS las CAs deben registrar los certificados que emiten en logs publicos auditables.
Si una CA emite un cert sospechoso, cualquiera lo detecta. Surgio en 2013 directamente como respuesta al caso DigiNotar y otros similares.

4. Las dos CAs de Fabric: identidad y TLS

Por qué se separan y qué consecuencias tiene

Dos problemas, dos soluciones

En Fabric (y en muchos sistemas) hay dos preguntas a resolver:

Problema 1 — Identidad: ¿quién firmo esta transacción?

Problema 2 — Comunicacion: ¿esta conexion es segura y entre quién?

Cada problema requiere su propia infraestructura criptografica:

- Identidad → CA de identidad y certificados de identidad
- Comunicación → TLS CA y certificados TLS

Comparativa: CA de identidad vs TLS CA

Aspecto	CA de identidad	TLS CA
¿Qué autoriza?	Quién firma transacciones	Quién abre conexiones de red
¿Dónde actúa?	Capa de aplicación (chaincode, lifecycle)	Capa de transporte (gRPC + TLS)
¿Quién la usa?	Peers, admins, usuarios firmando datos	Peers y orderers cifrando comunicación
¿En qué carpeta del MSP?	cacerts/	tlscacerts/
¿Qué pasa si la comprometen?	CATASTROFE — emiten transacciones falsas	Grave — pueden interceptar conexiones
Frecuencia de uso	Cada transacción	Cada conexión gRPC

Analogía: la carta firmada en sobre certificado

Imaginate enviar una carta importante:

CA de identidad → tu firma manuscrita validada por notario

- Garantiza que TU has escrito esa carta
- Sin firma, cualquiera podría haberla escrito

TLS CA → el sobre cerrado y certificado por correos

- Garantiza que la carta llega cerrada al destinatario correcto
- Sin sobre, cualquiera la lee por el camino

Las dos cosas son necesarias y resuelven problemas distintos.

Una carta firmada en sobre abierto -> cualquiera la lee

Una carta sin firmar en sobre cerrado -> ¿quién la escribió?

¿Por qué separar las dos CAs?

Aislamiento de daños — si comprometen una, la otra sigue protegiendo

Diferentes ciclos de vida — los TLS rotan mas a menudo que los de identidad

Diferentes politicas de gestion:

- CA de identidad: offline, en boveda, uso minimo

- TLS CA: online, accesible para emitir/renovar TLS

Diferentes responsables:

- CA de identidad: equipo de seguridad / compliance

- TLS CA: equipo de operaciones / DevOps

Flujo de una transaccion: dónde actúa cada CA

Cuando un peer del Hotel envia una transaccion al peer de la Cafeteria:

1. Conexion TCP segura (TLS):

El peer Hotel se conecta al peer Cafeteria por gRPC

Se establece TLS — los certificados TLS se validan con TLS CA

Si el cert TLS no esta firmado por una TLS CA conocida -> conexion rechazada

2. Envio de transaccion firmada (identidad):

El peer Hotel firma la transaccion con su clave privada de identidad

El peer Cafeteria valida la firma con la CA de identidad del Hotel

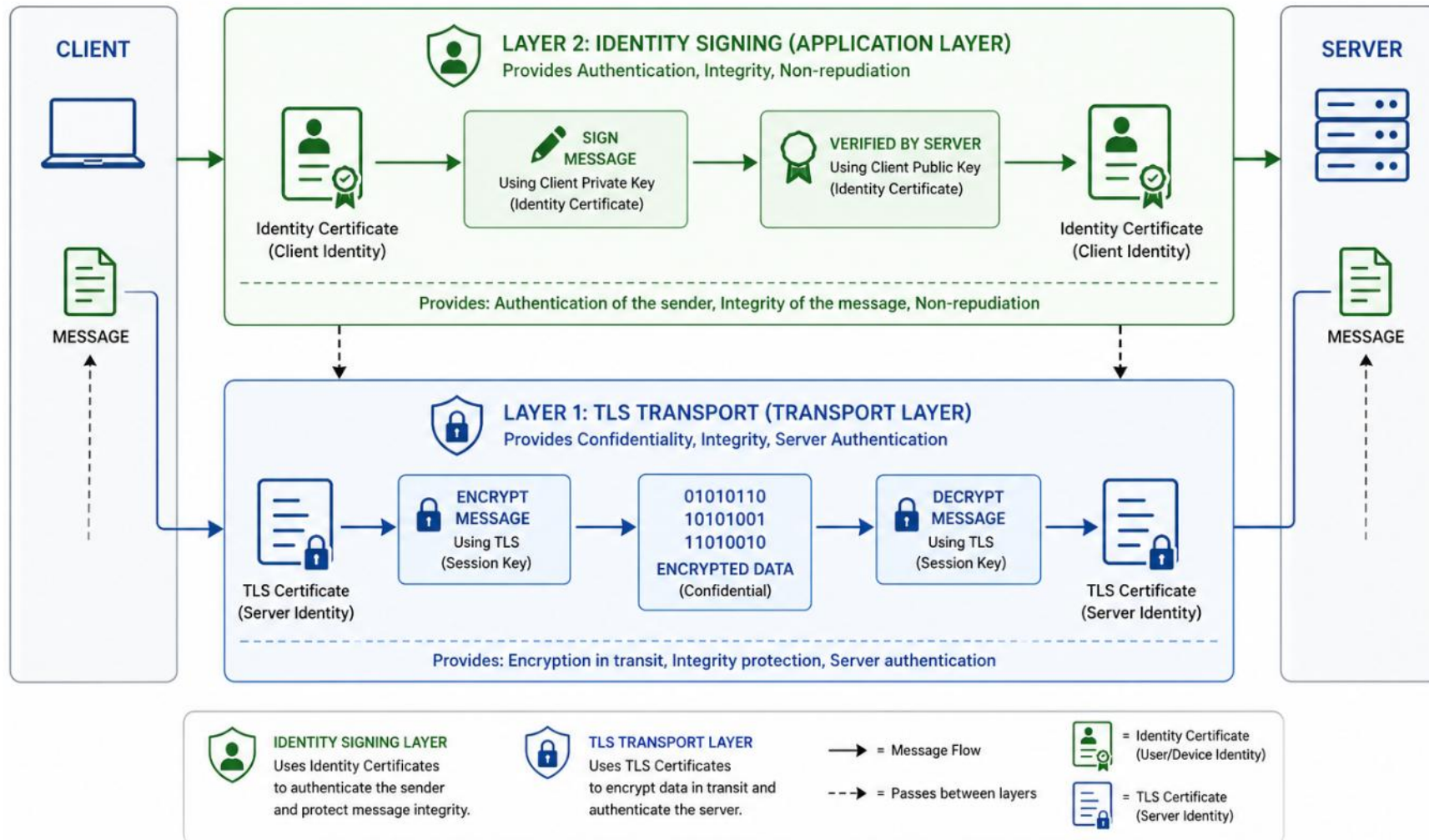
Si la firma no es valida -> transaccion rechazada

Sin las dos CAs, la transaccion no llega o no se valida.

Las dos capas de seguridad

LAYERED SECURITY: TRANSPORT + IDENTITY

Message flows through both layers for secure communication and verified identity



5. El MSP en Fabric

Membership Service Provider — la pieza que conecta todo

¿Qué es el MSP?

MSP = Membership Service Provider

Es el componente de Fabric que responde a:

- ¿Esta identidad pertenece a esta organización?

- ¿Que tipo de identidad es (peer, admin, client, orderer)?

- ¿Tiene los atributos necesarios para esta operacion?

Físicamente: una CARPETA con varios archivos de certificados

Lógicamente: la 'guia' que dice quién puede hacer qué en la red

Estructura del MSP

misp/

- cacerts/ — Cert raíz de la CA de identidad de la org
- tlscacerts/ — Cert raíz de la TLS CA
- keystore/ — Clave privada de ESTA identidad (privado)
- signcerts/ — Cert público de ESTA identidad
- intermediatecerts/ — Certs intermedios de la CA (opcional)
- crls/ — Listas de revocacion (opcional)
- config.yaml — Configuración de NodeOUs (roles)

Dos tipos de MSP

MSP de identidad (Local MSP):

- Representa a UN nodo o usuario concreto

- Tiene clave privada en keystore/ (la del propio nodo)

- Ubicacion: peers/peer0.../msp/, users/Admin@.../msp/, etc.

- Lo usa el peer/admin/user para firmar

MSP de organización (Channel MSP):

- Representa a la ORGANIZACION en su conjunto

- NO tiene clave privada — solo certs raíz publicos

- Ubicacion: org1.example.com/msp/

- Es lo que se referencia en configtx.yaml (campo MSPDir)

- Lo usan los demas peers para validar a miembros de esta org

config.yaml: NodeOUs

Define cómo se identifican los TIPOS de identidad dentro de una org

Hay 4 tipos estándar: client, peer, admin, orderer

Cada tipo se identifica por una OU (Organizational Unit) en el cert X.509

Sin NodeOUs, todos los miembros de la org son iguales — no hay distinción entre admins y usuarios normales

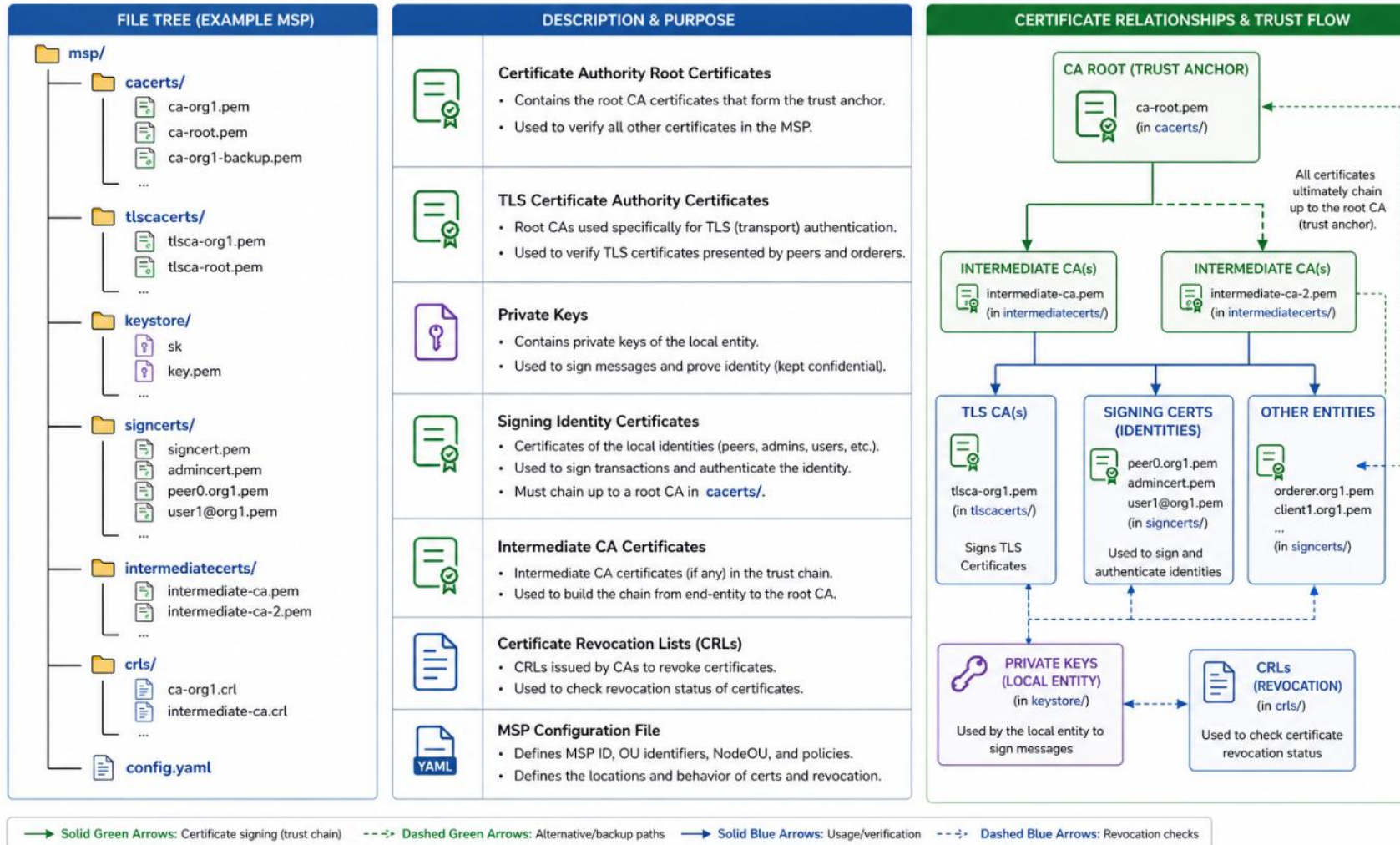
Con NodeOUs, el chaincode puede saber si quien le habla es un cliente, un peer, un admin o un orderer, y aplicar permisos diferentes

Por eso en cryptogen ponemos: EnableNodeOUs: true

Estructura completa del MSP en Fabric

MSP (MEMBERSHIP SERVICE PROVIDER) DIRECTORY STRUCTURE

Complete MSP Folder Layout and Certificate Relationships



Key Point: Certificates in **signcerts/**, **tlscacerts/**, and **intermediatecerts/** must chain up to a root CA in **cacerts/** to be trusted by the MSP.

6. Cómo se valida una transacción

El flujo completo paso a paso

Setup previo (lo que ya esta en su sitio)

Antes de cualquier transaccion, en el canal hay configurado:

MSP de Org1 con su cert raíz de identidad y de TLS

MSP de Org2 con sus certs raíz

MSP del Orderer con sus certs raíz

Cada peer tiene SUS claves privadas y SU MSP local

Las orgs YA se han intercambiado los certs raíz publicos

Flujo: Alice (Hotel) emite puntos a un cliente

1. Alice crea la propuesta de transaccion en su app cliente
2. La app firma la propuesta con la clave privada de Alice (identidad)
3. La app abre conexion TLS al peer del Hotel
4. El peer Hotel recibe la propuesta:
 - TLS valida la conexion TCP usando TLS CA
 - Verifica firma de Alice usando MSP del Hotel (CA de identidad)
 - Comprueba que Alice tiene rol 'admin' (NodeOU + atributos)
5. El peer Hotel ejecuta el chaincode (simulacion)
6. El peer Hotel firma la respuesta con SU clave privada
7. La app envia la propuesta endorsada al orderer:
8. El orderer ordena en bloques y los distribuye a todos los peers
 - Conexion TLS con orderer (validada con TLS CA del Orderer)
 - Orderer verifica las firmas de los endorsers
9. Cada peer valida el bloque:
 - Conexion TLS

Cuántas verificaciones hay en una transacción?

Por cada propuesta de transaccion que llega a un peer:

- Validacion del TLS de la conexion → TLS CA

- Validacion del cert de identidad de quien envia → CA de identidad

- Verificacion de la firma de la transaccion → clave publica del firmante

- Verificacion de cada firma de endorsement (por org) → CA de cada org

- Verificacion del cert del orderer cuando llega el bloque → TLS CA del orderer

Sumando: 4-6 verificaciones criptograficas por transaccion

Cada una usa una CA distinta (TLS o identidad) y tarda microsegundos

Por eso el rendimiento de Fabric depende mucho de la CPU y de la rapidez con la que valida firmas

Whitfield Diffie y Martin Hellman: los padres de la cripto moderna

En 1976, Diffie y Hellman publicaron 'New Directions in Cryptography'

Inventaron el concepto de criptografía de clave PÚBLICA

Hasta ese momento, toda la cripto era simétrica — necesitabas compartir la clave

Su idea revolucionó todo: comunicación segura entre desconocidos

Sin Diffie-Hellman no existiría:

- HTTPS (cada vez que entras en una web)
- Cualquier cert X.509 (incluido los de Fabric)
- Bitcoin, Ethereum, Hyperledger, blockchain en general
- Las firmas digitales tal como las conocemos

Recibieron el Premio Turing en 2015 (el Nobel de informática) por este trabajo.

Curiosidad: el algoritmo RSA llegó un año después (1977) basándose en sus ideas.

1. En un consorcio Fabric, ¿quién debería operar las CAs? ¿Cada org la suya, una neutra, ambas?
2. Si comprometen la CA del Hotel, ¿que pueden hacer los atacantes en la red? ¿Y si comprometen la TLS CA?
3. ¿Es razonable que la clave privada de la CA raíz se almacene en una bóveda física con HSM? ¿O es excesivo?
4. Si una org abandona el consorcio, ¿que pasa con los certs que emitio? ¿Como se invalidan?
5. ¿Tiene sentido que Fabric soporte certificados intermedios cuando la mayoría de proyectos no los usa?

Respuestas al debate (1/2)

1. Quién debería operar las CAs:

Cada org SU propia CA — es la solución estándar y más autónoma

Una CA central neutral sería más simple pero crea un punto de fallo

Lo habitual: cada org opera la suya y se intercambian certs raíz

El orderer puede tener su propia CA dedicada (para mayor aislamiento)

2. Si comprometen la CA del Hotel:

Atacantes pueden emitir certs falsos firmados por la CA del Hotel

Pueden firmar transacciones como si fueran cualquier miembro del Hotel

Pueden suplantar al admin del Hotel y aprobar chaincodes

Si comprometen la TLS CA: pueden interceptar conexiones (MITM)

- pero NO pueden firmar transacciones válidas

- el daño es mucho menor pero hay que rotar igualmente

3. CA raíz en bóveda con HSM:

Es absolutamente razonable, no excesivo, en proyectos serios

Los HSM evitan que la clave salga del hardware (ni siquiera el admin la ve)

Coste: ~10-30k EUR por HSM, pero protege un activo de millones

En aula no usamos HSM — pero hay que saberlo para el día que se haga real

Respuestas al debate (2/2)

4. Org que abandona el consorcio — que hacer con sus certs:

Tecnicamente: no se pueden borrar los datos del ledger (inmutabilidad)

Lo que se hace: revocar todos los certs emitidos por su CA

Generar nueva CRL (Certificate Revocation List) y distribuirla a los MSPs

Eliminar la org del MSP del canal (config update)

Sus transacciones historicas siguen siendo verificables (eran validas en su momento)

Pero NO puede emitir nuevas transacciones — su MSP ya no esta en el canal

5. ¿Tiene sentido los certs intermedios en Fabric?

Si, tiene mucho sentido en producción seria

Permite mantener la raíz offline y proteger el activo mas critico

Si comprometen el intermedio, lo revocas y emites otro sin tocar la raíz

En cryptogen no se usa por simplicidad (cadena de 1 nivel)

Fabric CA si lo soporta nativamente — config 'intermediate CA mode'

En proyectos enterprise serios es la norma, no la excepcion

Inspeccionar un certificado X.509 real

Ejercicio practico para fijar conceptos.

1. Usa la red FidelityChain levantada (o cualquier red Fabric)

2. Localiza un cert de un peer:

```
$HOME/proyecto-fidelitychain/network/crypto-config/  
peerOrganizations/hotel.fidelitychain.com/  
peers/peer0.../msp/signcerts/cert.pem
```

3. Inspecciona el cert con openssl:

```
openssl x509 -in cert.pem -text -noout
```

4. Identifica:

- Subject: ¿quién es el titular?
- Issuer: ¿quién lo emitio? ¿Coincide con la CA raíz de la org?
- Validity: ¿cuando caduca?
- Subject Alternative Names: ¿estan los hostnames y SANs correctos?
- Public Key: ¿que algoritmo usa? (ECDSA, P-256...)

5. Compara con el cert raíz (cacerts/ca-cert.pem). ¿En que se parecen y diferencian?

Tiempo: 30 minutos. Trabajar en parejas.

Repaso final

1. ¿Que es un certificado X.509 y que campos tiene?
2. ¿Que es una CA y por que confiamos en ella?
3. ¿Que es un cert raíz y por que se llama raíz?
4. ¿Que es la cadena de confianza y para que sirve?
5. ¿Por que Fabric usa dos CAs distintas? Nombra cada una y su funcion.
6. ¿Que es un MSP? Tipos y estructura de carpetas.
7. ¿Que son los NodeOUs y para que sirven?
8. ¿Cuantas verificaciones criptograficas hay en una transaccion?

Respuestas al repaso (1/2)

1. Certificado X.509:

Documento digital firmado que asocia identidad + clave publica

Campos: Subject, Issuer, Public Key, Validity, Extensions, Signature

Estandar internacional definido en RFC 5280

2. CA y por qué confiamos:

Certificate Authority — autoridad que emite certs firmandolos

Confiamos por: presencia en stores oficiales (publicas), acuerdos comerciales
o decision interna del consorcio (privadas/Fabric)

3. Cert raíz:

Cert auto-firmado al inicio del arbol de confianza

Subject e Issuer son la misma entidad

No tiene cert por encima — es el ancla de confianza

4. Cadena de confianza:

Secuencia de certs desde un cert hasta la raiz

Cada cert firmado por el de arriba

Para validar: recorrer la cadena hasta llegar a un raíz conocido

Respuestas al repaso (2/2)

5. Las dos CAs de Fabric:

CA de identidad — quien firma transacciones (capa aplicacion)

TLS CA — quien establece conexiones (capa transporte)

Estan separadas por seguridad y aislamiento de daños

6. MSP — Membership Service Provider:

Carpeta con certs que define quién pertenece a una org

Tipos: Local MSP (un nodo/usuario, con clave privada) y

Channel MSP (la org en si, sin clave privada)

Estructura: cacerts/, tlscacerts/, keystore/, signcerts/, etc.

7. NodeOUs:

Organizational Units que distinguen tipos de identidad: client, peer, admin, orderer

Permiten al chaincode aplicar permisos diferentes a cada tipo

Se activan con EnableNodeOUs: true

8. Verificaciones criptograficas en una transaccion:

TLS de la conexion (TLS CA)

Cert de identidad del firmante (CA de identidad)

Firma de la transaccion (clave publica del firmante)

Firma de cada endorsement (CA de cada org)

Cert TLS del orderer (TLS CA del orderer)

Total: 4-6 verificaciones por transaccion

Lo que ya sabes

Has entendido la base de identidad en Fabric

Resumen ejecutivo

PKI: la criptografia asimetrica + certificados firmados por una CA

CA: la autoridad cuya firma da validez a los certificados

Cert raíz: el ancla de confianza, auto-firmada, en la cima del arbol

Cadena de confianza: la secuencia firma-firma desde un cert hasta el raíz

Fabric usa DOS CAs separadas: identidad (firmar transacciones) y TLS (cifrar comunicacion)

MSP: la carpeta de certs que define una identidad o una organizacion en Fabric

NodeOUs: distinguen tipos de identidad (client, peer, admin, orderer)

Cada transaccion implica 4-6 verificaciones criptograficas — Fabric esta optimizado para ello

Ya tienes la base para entender Fabric CA, MSPs, lifecycle y operaciones avanzadas.

Anexo: Cert intermedio vs Cert end-entity

La diferencia clave: ¿quién puede firmar otros certs?

El campo clave: Basic Constraints

Todos los certificados X.509 tienen un campo llamado 'Basic Constraints'

Ese campo indica si el cert puede firmar OTROS certificados o no

CA: TRUE → este cert puede firmar otros (es una CA)

CA: FALSE → este cert NO puede firmar otros, solo se usa para si mismo

Es lo que distingue funcionalmente a un cert intermedio (CA: TRUE)

de un cert end-entity como el de un peer o usuario (CA: FALSE).

Tabla comparativa de los 3 tipos de cert

Aspecto	Cert raíz	Cert intermedio	End-entity
¿Auto-firmado?	Si (Subject = Issuer)	No (firmado por raíz)	No (firmado por CA)
Basic Constraints CA	TRUE	TRUE	FALSE
¿Puede firmar otros certs?	Si	Si	NO
¿Donde se almacena?	Offline, en HSM/boveda	Online, en servidor CA	En el nodo (peer, app)
Validez tipica	10-20 años	5-10 años	1-2 años
Para qué se usa	Anclar la confianza	Emitir end-entities	Firmar transacciones / TLS

Intermedio vs end-entity: la diferencia real

Aspecto comun:

Ambos son firmados por un cert superior

Ambos son archivos .pem identicos en formato

Ambos tienen Subject, Issuer, clave publica, validez...

Aspecto que los diferencia:

El intermedio tiene 'CA: TRUE' y la opcion 'keyCertSign' en keyUsage

El end-entity tiene 'CA: FALSE' y NO puede firmar otros certs

Es UN solo campo en el cert lo que cambia su comportamiento

Si un end-entity intentara emitir un cert:

1. End-entity (Alice) firma un cert para Bob
2. Bob presenta su cert a Carlos para validarlo
3. Carlos recorre la cadena: Bob -> firmado por Alice
4. Carlos comprueba el cert de Alice: CA: FALSE
5. RECHAZADO — un end-entity no puede ser CA

Por eso es seguro distribuir certs end-entity sin riesgo de cadena ilegítima

Como verlo en la practica con openssl

Inspecciona un cert de un peer (end-entity) en la red FidelityChain:

```
openssl x509 -in cert.pem -text -noout | grep -A 2 "Basic Constraints"
```

Resultado para un peer (end-entity):

```
X509v3 Basic Constraints: critical
```

```
CA:FALSE
```

Compara con la CA raíz de la org:

```
X509v3 Basic Constraints: critical
```

```
CA:TRUE
```

Esa unica linea (CA:TRUE vs CA:FALSE) marca toda la diferencia.

Es lo que protege la cadena de confianza de delegaciones no autorizadas.

En Fabric: ¿cuando ves cada tipo?

Con cryptogen (desarrollo, aula):

- 1 cert raíz por org → CA: TRUE (en cacerts/)

- Certs de peers, admins, users → CA: FALSE (end-entity)

- Solo 2 niveles, sin intermedios

Con Fabric CA en produccion seria:

- 1 cert raíz por org → CA: TRUE, offline en HSM

- 1+ certs intermedios → CA: TRUE, en servidor CA

- Certs de peers, admins, users → CA: FALSE

- 3+ niveles, con intermedios protegiendo la raíz

Como verificar si tu cert es CA o end-entity:

- `openssl x509 -in cert.pem -text -noout | grep -i 'CA:'`

- Si dice CA: TRUE → es una CA (raíz o intermedio)

- Si dice CA: FALSE → es un end-entity